Guida al Workflow di Scaffold

Un workflow neutrale, copia-e-adatta per lo sviluppo assistito da AI con Claude Code e Google Antigravity: come ogni parte viene generata e configurata, come funzionano le sue regole e a cosa serve.

Il workflow è nato da un lungo progetto ed è stato ripulito da tutto ciò che era specifico per esso. Ciò che resta è un insieme di file, regole e piccoli script da poter copiare in qualsiasi repository. Sostituisci i segnaposto `<PROJECT>`, `<area>`, `<build-cmd>` e `<test-cmd>`, mantieni la struttura.

Come leggere questa guida. Ogni sezione ha la stessa struttura: un riquadro **TL;DR**, la spiegazione dettagliata e un blocco finale di **Esempi pratici**. Gli esempi utilizzano un'applicazione web fittizia, Taskly (un server Python, un client web e un client mobile); gli output catturati provengono dai veri script eseguiti su `examples/demo`.

Documenti complementari. Il riassunto di una pagina con i link a ciascun file è `docs/WORKFLOW.md`. Tutto ciò che è verificabile viene controllato da `python scripts/selftest.py`. Questo PDF si ricompila con `python scripts/build_workflow_pdf_it.py`.

Sommario

1. Perché un workflow . . . 3
2. Struttura del repository . . . 5
3. CLAUDE.md e recupero mirato . . . 7
4. Il sistema di documentazione . . . 9
5. Ruoli dei subagenti e routing . . . 11
6. Readiness Gate . . . 13
7. Escalation basata su evidenze . . . 15
8. Punteggio di necessità, soglia di autonomia, ledger e agent_audit . . . 16
9. Consistency gate: file di specifica, deriva, divergenze . . . 18
10. Audit della documentazione . . . 20
11. Audit della CI . . . 21
12. Sincronizzazione multi-operatore . . . 22
13. Scrivere regole come istruzioni positive . . . 24
14. Bootstrap passo dopo passo . . . 25
15. Manutenzione e metriche . . . 27

1. Perché un workflow

TL;DR Gli agenti AI falliscono in modi prevedibili: rileggono tutto, tirano a indovinare se la task è vaga, sprecano il modello costoso su lavoro banale, lasciano disallineare documentazione e piattaforme, ed entrano in conflitto tra loro. Il workflow trasforma ogni tipo di fallimento in una regola, un file e uno script che lo controlla.

I fallimenti e il meccanismo che risponde a ciascuno

Fallimento	Meccanismo	Dove si trova
Gli agenti leggono file interi e bruciano token	Recupero mirato: un solo indice, grep e poi lettura di uno slice	`CLAUDE.md` sez. 1, `docs/progress.md`
Una task vaga produce un diff errato	Il planner la scompone in Work Order; il Readiness Gate rifiuta le task non pronte	`.claude/agents/planner.md`, skill `work-order`
Il modello costoso esegue compiti banali	Routing dei ruoli, punteggio di necessità con soglia di autonomia, ledger	`CLAUDE.md` sez. 3, `docs/agents/ledger.txt`
Gli agenti dichiarano DONE senza prove	Stato basato su evidenze: DONE richiede un comando che passa	file degli agenti, `CLAUDE.md` sez. 3
La documentazione invecchia e nasconde regole	Una scheda per area con sezioni fisse `Open work` e `Rules`, verificata da uno score	`docs/progress/`, `scripts/docs_audit.py`
Le piattaforme divergono silenziosamente	Registro di ciò che è promesso e un gate che elenca le divergenze	`docs/spec/`, `scripts/consistency.py`
La CI spreca minuti preziosi	Regole su timeout, concorrenza e branch ignorati, verificate da audit	`scripts/ci_audit.py`
Agenti paralleli collidono o spariscono	Protocollo di sync: un solo scrittore per file, inbox, stati, consegne, review	`SYNC.md`, `sync/`, `tools/sync_*.py`
Le regole vengono ignorate o fraintese	Regole scritte come istruzioni positive, ritirate cancellandole	`CLAUDE.md` sez. 2

Il principio: separare decidere da eseguire

Un modello di punta costa molto sui token di output ed eccelle nel ragionamento logico, quindi deve produrre decisioni e piani sintetici. Un modello economico è ottimo per scrivere molto codice partendo da un ordine blindato. Il router sceglie l'agente in base a **quanto la task è già specificata**, non a quanto sembra grande. Una richiesta di una riga può essere vaga e pericolosa; un rename su dieci file può essere meccanico e sicuro.

Tutto è verificabile

Una regola che nessuno può verificare decade. Ogni componente ha il suo script: `docs_audit.py` misura i documenti, `consistency.py` confronta i fatti tra file diversi, `agent_audit.py` analizza il ledger, `ci_audit.py` esamina il workflow di CI, `sync_check.py` legge lo stato di sincronizzazione. `selftest.py` esegue tutti i controlli sul progetto demo e rompe intenzionalmente la demo per dimostrare che ogni script rileva le anomalie.

Esempi pratici

Esempio 1: una richiesta vaga, con e senza il workflow

Richiesta: "migliora la ricerca". Senza workflow un coder economico inizia a modificare: nove file toccati, circa 40k token, un risultato che nessuno può verificare. Con il workflow, la richiesta va a `planner`, che produce tre Work Order (un parser di filtri, un ritocco all'indice, un chip UI), ciascuno con file, comando di test e agente designato. Due superano il Readiness Gate e vanno a `coder`; uno fallisce la domanda 5 (richiede una decisione di prodotto) e diventa una voce in `docs/open-decisions.md`.

Esempio 2: due agenti, un solo file
Due operatori hanno entrambi bisogno del file di build. Nel protocollo di sync, la divisione dei file nella sezione 2 di `SYNC.md` indica il proprietario di ogni file; il file di build condiviso è consentito solo con "una riga a testa, indicata nel Work Order". Nessun agente sovrascrive l'altro, e il diff di revisione rispetto al perimetro concordato lo dimostra.

Esempio 3: il costo che nessuno vede
Il ledger della demo mostra 12 esecuzioni: l'unica sessione di `deep` (88.000 token, 420 s) è stata registrata con necessità 2, e `agent_audit.py` lo segnala chiaramente: `expensive agent used 1 time(s) with necessity <= 3`. Senza il ledger la stessa spesa resta invisibile.

2. Struttura del repository

TL;DR Pochi percorsi fissi: note per gli agenti nella radice, definizioni di subagenti e skill sotto `.claude/`, documentazione sotto `docs/`, script di audit sotto `scripts/`, tool di sync sotto `tools/`, template del protocollo in `SYNC.md` e `sync/`, e una demo compilata sotto `examples/demo/`. Gli script accettano `--root DIR`.

L'albero dei file

```
<repo>/
  CLAUDE.md  AGENTS.md  GEMINI.md        note di sessione per Claude Code, agenti generici, Antigravity
  SYNC.md  sync/                         TEMPLATE protocollo sync (copia attiva su branch sync/<nome>)
    _template.md  operators.toml  inbox/README.md
  .claude/agents/                        planner, coder, mechanic, light, deep, research, ops
  .claude/skills/                        work-order, docs-loop, docs-map
  docs/progress.md                       l'unico indice
  docs/progress/<area>.md                una scheda per area: Open work, Rules (do not break)
  docs/ROADMAP.md  docs/open-decisions.md  docs/WORKFLOW.md
  docs/agents/ledger.txt                 esecuzioni dei subagenti, append only
  docs/spec/                             specs.toml config-keys.toml platforms.toml divergences.toml
                                         known.toml consistency.toml CONSISTENCY.md
  scripts/                               docs_audit consistency agent_audit ci_audit selftest
                                         build_workflow_pdf
  tools/                                 sync_hdr sync_check sync_watch
  .github/workflows/ci.yml  .ci-audit.toml
  examples/demo/                         "Taskly": un piccolo progetto compilato su cui girano gli script
```

Cosa si trova e dove

Posizione	Branch principale	Branch di sync
`CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, `.claude/`, `docs/`, `scripts/`	sì	ereditato dal main
`SYNC.md`, template di `sync/`, `tools/sync_*.py` (template)	sì: solo template	la copia attiva e compilata
`sync/<ID>.md`, `sync/inbox/<ID>.md`, `sync/wo/<ID>.md`, righe Log	mai	sì, soltanto qui

Convenzioni condivise dagli script

- Python 3.11 o successivo puro (`tomllib`), nessuna dipendenza esterna. Solo il generatore PDF richiede `reportlab`.
- `--root DIR` punta lo script su un'altra cartella di progetto; il valore predefinito è la cartella sopra `scripts/` (o `tools/`). I tool di sync leggono anche la variabile d'ambiente `SYNC_ROOT`.
- Exit code 0 indica esito positivo, 1 indica una segnalazione, 2 indica un errore d'uso. La CI può eseguire qualsiasi script e fidarsi dell'exit code.
- Nessuno script scrive sui tuoi documenti, tranne il tool esplicitamente richiesto (`sync_hdr.py` scrive un solo blocco; `docs_audit.py log` appende una riga).

Esempi pratici

Esempio 1: adattare la struttura a Taskly

Taskly colloca il server in `src/server/`, quindi `docs/progress/api.md` elenca `src/server/app.py` nella colonna del codice principale, e il blocco `[[match]]` in `docs/spec/consistency.toml` punta allo stesso file. Niente altro nella struttura è cambiato.

Esempio 2: un solo comando dimostra che il kit funziona

```
$ python scripts/selftest.py
selftest: 40/40 checks passed
```

Eseguilo dopo aver copiato il kit in un nuovo repository: compila tutti gli script, esegue ciascuno sulla demo e controlla che le copie deliberatamente corrotte della demo vengano intercettate.

Esempio 3: perché esiste examples/demo
Un template con segnaposto è difficile da valutare. La demo è un piccolo progetto completo con numeri reali (uno score di 717, due blocchi al gate, un ledger con un'esecuzione fallita): aprilo accanto ai tuoi file per vedere come si presenta "compilato". È anche la fixture su cui vengono testati gli script di audit.

3. CLAUDE.md e recupero mirato

TL;DR `CLAUDE.md` è il briefing caricato a ogni sessione: come trovare le informazioni, regole di ingaggio, routing degli agenti, invarianti, chiusura delle task e handshake di sync. Resta snello puntando a `docs/progress.md`. Gli agenti cercano con grep e leggono porzioni mirate, mai interi file.

Come generarlo

Copia `CLAUDE.md` dallo scaffold e sostituisci `<PROJECT>`, `<main-branch>`, `<build-cmd>` e `<test-cmd>`; elimina le righe della tabella di consultazione rapida che non si applicano. `AGENTS.md` replica la sezione 3 per i tool che leggono quel nome; `GEMINI.md` contiene solo ciò che differisce per Antigravity (la mappatura dei modelli e la richiesta sul nome del branch). In caso di discrepanza, vince `CLAUDE.md`.

Le sei sezioni

Sezione	Cosa contiene	Perché è presente
1 Recupero mirato	Tabella di consultazione rapida con comandi esatti	Un agente che conosce il comando non esplora alla cieca
2 Regole di ingaggio	Nomi branch, test selettivi, segreti, licenze, scrittura regole	Comportamento che deve essere identico in ogni sessione
3 Subagenti e routing	Ruoli, Readiness Gate, score di necessità, ledger, escalation	Separare decidere da eseguire
4 Invarianti globali	Fatti sempre veri per l'intero progetto	Vincoli che nessuna scheda d'area governa da sola
5 Chiusura delle task	Checklist in sei passi eseguita prima di dichiarare una task completata	Documenti e controlli restano sempre allineati al codice
6 Sync multi-operatore	Handshake, consegna, inbox, heartbeat, sign-off	Più agenti possono unirsi con una sola istruzione

Recupero mirato, in pratica

```
grep -n -A 15 '^## Open work' docs/progress/<area>.md      # cosa resta da fare in un'area
grep -n -A 20 '^## Rules'     docs/progress/<area>.md      # le regole vincolanti di un'area
grep -n -m5 '<symbol>' <file>                              # codice in file grandi, poi Read offset/limit
```

La ragione è matematica. Un file sorgente da 350 KB contiene circa 90.000 token; la sezione di cui un agente ha bisogno è di solito lunga 40 righe. Leggere solo quella porzione costa circa l'1%. L'indice (`docs/progress.md`) esiste affinché l'agente non debba mai cercare a tentativi il comando di ricerca.

Mantenerlo snello

- Le regole specifiche di progetto vanno nella sezione `## Rules (do not break)` della scheda d'area, mai in `CLAUDE.md`; lascia solo una riga di puntatore.
- `docs_audit.py score` calcola `CLAUDE.md` più `docs/progress.md` come costo sempre caricato; la skill `docs-loop` lo riduce ciclo dopo ciclo.
- I segnaposto tra parentesi angolari vengono sostituiti una sola volta; dopodiché il file non contiene più testi da template.

Esempi pratici

Esempio 1: uno slice invece di un'intera scheda

```
$ grep -n -A 3 '^## Open work' docs/progress/api.md
14:## Open work
15-
16-- [ ] Add pagination to `/search` (new spec item and revision 3).
```

Circa 120 token contro i circa 4.000 dell'intera scheda; lungo un'intera giornata di lavoro la differenza rappresenta l'intero valore dell'indice.

Esempio 2: spostare una regola fuori da CLAUDE.md
`CLAUDE.md` conteneva un paragrafo sui touch target del client mobile. La skill `docs-loop` lo ha spostato in `## Rules` dentro `docs/progress/web.md`, lasciando solo un puntatore. Lo score di `docs_audit.py` è sceso da 5.210 a 4.480: ogni sessione che non tocca il client mobile ha smesso di pagare per quel contesto.

Esempio 3: un secondo tool, le stesse regole
Un collega apre il repository in Antigravity. `GEMINI.md` gli impone di proporre un nome branch breve prima di editare e mappa i ruoli su `pro`, `flash` e `flash_lite`; i comandi di recupero, il Readiness Gate e la checklist di chiusura sono identici, così le review hanno lo stesso aspetto qualunque tool abbia prodotto il diff.

4. Il sistema di documentazione

> **TL;DR** Un solo indice (`docs/progress.md`), una scheda per ciascuna area (`docs/progress/<area>.md`) con sezioni fisse, una roadmap per livelli e un elenco di decisioni aperte. I documenti descrivono lo stato attuale, non la storia; `docs_audit.py` li mantiene collegati e concisi.

L'indice

`docs/progress.md` è l'unico file a cui punta `CLAUDE.md`. Contiene una tabella delle aree (file, perimetro, codice principale), un elenco "quale sezione aprire", una breve lista di documenti di riferimento e i milestone attivi. Qualsiasi altro documento deve essere raggiungibile da una riga dell'indice; una scheda non collegata è un **orfano** e comporta una penalità.

La scheda d'area

Copia `docs/progress/core.md` per ogni area. Tre sezioni fisse, in questo ordine:

- `## How it is today`: un `###` per ciascun modulo, con i file e i simboli che vale la pena cercare, e il grep per trovarli. Solo stato attuale; nessuna data, nessun "abbiamo fatto".
- `## Open work`: voci con checkbox, ciascuna con il nome del file da toccare e il controllo che ne prova il completamento. Una voce chiusa viene rimossa, non lasciata spuntata per sempre.
- `## Rules (do not break)`: vincoli per mantenere l'area funzionante, scritti in positivo su cosa fare. Aggiunti quando emerge un vincolo, cancellati quando non è più vero.

Limite dimensionale: circa 120 righe. Una scheda più lunga va suddivisa per argomento (`<area>-<tema>.md`), collegata dalla scheda padre e dall'indice.

La roadmap

`docs/ROADMAP.md` raggruppa le attività in **livelli**; un livello inizia solo al completamento del precedente. Ogni riga ha sette colonne:

Colonna	Significato
ID	identificatore stabile (`R1`, `R2`...) usato in commit, work order e schede d'area
Pri	1 = massima priorità ... 5 = facoltativo
Stato	`open`, `active`, `blocked`, `suspended`
Obiettivo	una frase: cosa sarà vero quando il task sarà finito
Requisito	la condizione verificabile di completamento: un comando, un test, un numero
Dettagli	dove si trovano gli appunti di lavoro
Dopo	gli ID che devono terminare prima, oppure `-`

Le voci completate lasciano il file (git conserva la cronologia). Le nuove idee vengono aggiunte in fondo come righe `R<n>` fino alla promozione in un livello. Le questioni che richiedono una decisione del responsabile vivono in `docs/open-decisions.md`, una sezione per area, e vengono rimosse una volta decise (il risultato passa nella scheda d'area).

Esempi pratici

> **Esempio 1: chiusura di una task**
> La paginazione per `/search` è integrata. In `docs/progress/api.md` la voce lascia `## Open work`; compare una nuova regola sotto `## Rules`: "`/search` accetta `page` e `page_size`; `page_size` ha un limite massimo di 100". `docs_audit.py score` segnala ancora zero penalità, a conferma della corretta chiusura.

Esempio 2: una riga della roadmap che indica quando è completata

```
| ID | Pri | Status | Goal | Requirement | Details | After |
| R7 | 2 | open | Search filters on both clients (M) | gate: 0 blockers for D01 | web | R3 |
```

Il requisito è un comando che chiunque può eseguire. "After R3" segnala al planner di non avviare R7 prima dei lavori sul release gate.

Esempio 3: suddividere una scheda troppo lunga

`docs/progress/api.md` ha raggiunto 190 righe e `docs_audit.py` ha applicato 200 punti di penalità. Un ciclo con `docs-loop` ha separato la parte di auth in `api-auth.md`, collegandola dall'indice e preservando le informazioni (spostate, non perse). Lo score è calato della penalità e il costo sempre caricato non è cresciuto.

5. Ruoli dei subagenti e routing

> **TL;DR** Sette ruoli con modelli, tool e limiti prefissati: `planner` (pianifica), `coder` (scrive da un ordine), `mechanic` (modifiche a rischio zero), `light` (giudizio tecnico, fix contenuti, docs verificate), `deep` (un solo problema critico isolato), `research` (ricerca in sola lettura), `ops` (comandi shell già decisi). Il router sceglie in base al grado di specificazione della task.

I ruoli

Agente	Modello / effort	Tool	Cosa fa	Cosa NON fa
`planner`	Opus, medium	sola lettura	Trasforma task vaghe in Work Order	scrivere codice
`coder`	Haiku, high	read, edit	Scrive codice da un ordine preciso	progettare, risolvere ambiguità
`mechanic`	Haiku, low	read, edit	Rename, formato, template, spunte avanzamento	prendere decisioni logiche
`light`	Sonnet, medium	read, edit	Piccoli fix, documentazione verificata sul codice	lavori lunghi o vaghi
`deep`	Opus, xhigh	read, edit	Un solo problema critico isolato con riproduzione	implementare intere feature
`research`	Haiku, low	sola lettura	Ricerche su molti file, sintesi in 5-10 righe	modificare qualsiasi file
`ops`	Haiku, low	solo Bash	Esegue comandi già decisi, riporta PASS/FAIL	decidere, correggere, kill, merge

Un file di definizione agente

Ciascun ruolo è un file Markdown in `.claude/agents/` con front matter e un corpo sintetico. Il front matter costituisce il contratto:

```
---
name: coder
description: Writes code EXCLUSIVELY from a precise, exhaustive Work Order. Makes no design decisions.
model: haiku          # opus | sonnet | haiku
effort: high          # low | medium | high | xhigh
tools: Read, Edit, Write, Grep, Glob
---
You are the code-writing arm of <PROJECT>. ...
## Closing states      DONE | BLOCKED: <reason> | FAILED x2
```

Il campo `description` è ciò che l'agente principale legge durante la scelta, quindi definisce trigger e vincoli. Il corpo ripete i limiti come regole positive e si chiude con gli stati finali (sezione 7). Per creare il tuo set: copia la cartella, sostituisci `<PROJECT>` e `<test-cmd>`, e adatta i modelli se il tuo account ne offre altri.

Routing

1. La task è già specificata? Applica il Readiness Gate (sezione 6). Sì: `mechanic` (pura meccanica), `coder` (codice dall'ordine), `light` (richiede un minimo di giudizio tecnico).
2. No: `planner` formula prima i Work Order. Una task vaga non va mai direttamente a un esecutore, e mai a `deep`.
3. Un bug isolato con riproduzione certa che gli agenti economici hanno fallito due volte: `deep`, con necessità dichiarata (sezione 8).
4. Cercare informazioni su molti file: `research`. Eseguire comandi già concordati: `ops`.

Mappatura su Antigravity

`AGENTS.md` e `GEMINI.md` mappano gli stessi ruoli su Gemini: `planner` e `deep` su `pro`, `coder` e `light` su `flash`, `mechanic` e `ops` su `flash_lite`, `research` sul subagente di ricerca nativo.

Esempi pratici

Esempio 1: due segnalazioni di bug, due percorsi diversi
"Il pulsante Salva è spostato di 4 px su mobile": un solo file, un valore CSS, un test visivo. Il gate passa, `light` risolve. "Il test di ricostruzione indice a volte fallisce": nessuna riproduzione, causa ignota. Va a `planner`, che richiede un ciclo per isolarlo; solo dopo un comando fallito e due tentativi a vuoto interviene `deep`.

Esempio 2: l'heartbeat sull'agente economico
L'heartbeat di un operatore comprende fetch, inbox, ack. Se eseguito dall'agente principale rilegge l'intera conversazione a ogni ciclo; eseguito da `ops` con un prompt di tre righe costa poche migliaia di token. Il file di `ops` vieta `kill`, `reset --hard` e i merge, così delegare task shell non crea rischi.

Esempio 3: una sintesi invece di quaranta file
"Chi legge `page_size`?" va a `research`, che risponde in sei righe: `src/server/app.py:14` lo legge, il registro `docs/spec/config-keys.toml` lo elenca, nessun altro. Il contesto principale riceve sei righe anziché i contenuti completi dei file.

6. Readiness Gate

TL;DR Cinque domande sì/no prima che una task passi a `mechanic`, `coder` o `light`. Cinque SÌ: pronta. Anche solo un NO: la task va a `planner`, che la rende pronta o la scompone. Il gate permette a un modello economico di lavorare senza tirare a indovinare.

Le cinque domande

#	Domanda	Un NO di solito significa
1	**File e funzioni**: i file e le funzioni da toccare sono indicati esplicitamente?	l'agente dovrebbe cercare e scegliere arbitrariamente
2	**Determinismo**: due sviluppatori produrrebbero lo stesso diff (nessuna scelta di design aperta)?	una decisione architetturale è nascosta nell'ordine
3	**Criterio verificabile**: esiste un comando deterministico per la verifica?	"fatto" resterebbe un'opinione soggettiva
4	**Perimetro limitato**: circa 3 file o meno?	la task va spezzata in passaggi più piccoli
5	**Zero blocchi**: indipendente da decisioni ancora in sospeso?	qualcuno deve prima prendere una decisione

La regola di esito

- **5 SÌ su 5**: PRONTA. Scegli l'esecutore: `mechanic` se puramente meccanica, `coder` per codice dall'ordine, `light` se serve un minimo di giudizio tecnico.
- **Anche solo un NO**: NON PRONTA. `planner` risolve l'ambiguità ed emette uno o più Work Order, ciascuno dei quali deve superare il gate.
- `deep` non è il ripiego per un gate fallito; richiede una procedura di riproduzione e una necessità formale dichiarata.

Il Work Order

La skill `work-order` contiene il template standard. Il planner compila un blocco per ciascun passaggio; l'esecutore non deve mai domandarsi cosa si intendesse.

```
### WORK ORDER: WO-12
- **Objective**: reject an empty value in the tag filter.
- **Files/functions to touch**: src/server/search.py:parse_filters(); tests/test_search.py
- **Required change**: when a filter value is empty ("tag="), raise FilterError("empty value").
- **Verification command**: python -m pytest tests/test_search.py -k filters
- **Expected result**: exit 0, 8 passed
- **Assigned agent**: coder
```

Dimensione: un Work Order richiede 45 minuti o meno (ideale 15). Lavori più ampi vengono suddivisi, un ordine per ciascun passaggio.

Esempi pratici

Esempio 1: un Work Order che il gate rifiuta

```
Objective: improve search relevance.
Files: src/server/search.py (and whatever else is needed)
Change: tune the ranking.
Verify: check that results look better.
```

D1 NO ("whatever else"), D2 NO (quale accordatura?), D3 NO ("look better" non è un comando), D5 NO (la logica di ranking non è decisa). Quattro NO: va a `planner`, che chiede al responsabile di decidere la policy e poi scrive due ordini verificabili con `pytest -k ranking`.

Esempio 2: la stessa task, corretta

Il WO-12 sopra cita una funzione e un file di test, indica il comportamento esatto e fornisce un comando con esito atteso. Cinque SÌ, assegnato a `coder` (Haiku). Completato in 6 tool call e 60 secondi, stato `DONE`.

Esempio 3: il gate si calibra da solo
Le righe del ledger annotano il punteggio al gate. Il ledger della demo evidenzia che 2 esecuzioni su 7 con gate <= 2 non terminano con `DONE` (28,6%), e `agent_audit.py` stampa "gate too optimistic". La soluzione è stata applicare la domanda 2 con maggior rigore: ordini con "porta il pannello" senza nomi di file hanno smesso di passare.

7. Escalation basata su evidenze

TL;DR Ogni agente termina esattamente con uno stato: `DONE` (il comando di verifica è passato con successo), `BLOCKED: <motivo>` (manca una decisione; torna a `planner`), oppure `FAILED x2` (due tentativi falliti; sale di un livello). Le affermazioni senza prove non vengono accettate.

I tre stati

Stato	Significato	Cosa accade dopo
`DONE`	Il comando di verifica del Work Order è passato; segue un riepilogo diff di 2-3 righe	Integrazione; registrazione dell'esecuzione nel ledger
`BLOCKED: <motivo>`	Specifica ambigua, firma non corrispondente o decisione mancante	Ritorno a `planner`, che chiarisce o scompone l'ordine
`FAILED x2`	La verifica fallisce ancora dopo due tentativi di correzione	Escalation di un livello: da coder a light, da light a deep (con punteggio di necessità)

Perché proprio questi stati

- **Evidenze prima delle affermazioni.** DONE è vincolato a un comando eseguibile. "Credo funzioni" non è uno stato valido.
- **BLOCKED è economico e onesto.** Gli agenti coder hanno il divieto di inventare soluzioni; fermarsi subito al punto di blocco costa molto meno di un diff sbagliato.
- **Due tentativi, poi escalation.** Un terzo tentativo identico raramente ha successo. `FAILED x2` traccia cosa è stato provato per evitare che il livello successivo lo ripeta; `deep` rifiuta richieste prive di questa cronologia.
- **La scala è corta.** mechanic e coder scalano su light; light scala su deep. Qualsiasi cosa richieda una decisione scende lateralmente su planner, non sale.

Gli stati alimentano anche il ledger (sezione 8): la colonna di esito accetta `DONE`, `BLOCKED`, `FAILED x2` ed `ESCALATED`.

Esempi pratici

Esempio 1: coder fallisce due volte, light conclude

Il WO-2 (scrittura CSV) è andato a `coder`; il test continuava a fallire sulle virgolette. Dopo due tentativi ha riportato `FAILED x2` con l'asserzione fallita. `light` l'ha preso, ha letto il comportamento effettivo della libreria standard, ha corretto le virgolette e ha chiuso `DONE` in 180 s. Righe nel ledger: `coder ... FAILED x2`, poi `light ... DONE`.

Esempio 2: BLOCKED impedisce di tirare a indovinare

Il WO-4 recitava "porta il pannello dei filtri". Il coder ha risposto `BLOCKED: decision needed - which filters, and is the date range a picker or two fields?`. Una volta che il responsabile ha scelto, `planner` lo ha diviso in tre ordini; nulla è stato costruito su ipotesi arbitrarie.

Esempio 3: un DONE senza prove viene respinto

Un agente ha riportato `DONE` per un rename senza aver mai lanciato la build. Il revisore l'ha eseguita: due file usavano ancora il vecchio nome. La regola secondo cui DONE esige un comando passato ha trasformato il rifiuto in un messaggio di una riga senza discussioni.

8. Punteggio di necessità, soglia di autonomia, ledger e agent_audit

> **TL;DR** L'uso del modello costoso richiede un punteggio dichiarato da 1 a 5 con una motivazione esplicita. Con la soglia di autonomia a 4, i punteggi 4-5 attivano il cambio senza chiedere, mentre 1-3 richiedono conferma. Ogni esecuzione è registrata in `docs/agents/ledger.txt`, e `agent_audit.py` trasforma i dati in indicazioni di calibrazione.

Il punteggio

N	Significato
1	Il modello economico può farlo; richiesto solo per comodità
2	Quasi certamente il modello economico è sufficiente; quello costoso risparmia al massimo un turno
3	Probabile un errore o due turni con il modello economico: parte prima `planner`
4	Il modello economico ha già fallito, oppure l'errore ha un costo elevato
5	Quasi una necessità: l'errore è irreversibile, oppure l'alternativa è bloccarsi

Una richiesta dichiara **N su 5, l'unica motivazione, e il moltiplicatore di costo stimato rispetto al modello economico** (ad esempio "circa 5x"). Esistono solo due motivazioni valide: **fallimento pregresso** ("un modello precedente ha già fallito, `FAILED x2`") oppure **alto costo dell'errore** ("risolvere o rifare a causa di un errore è costoso").

La soglia di autonomia

`CLAUDE.md` sezione 3 contiene una sola riga: **autonomy threshold = 4**. Con N=4 o 5 l'agente passa al modello costoso senza chiedere, indicando punteggio e motivazione in chat. Con N=1-3 non lo fa. Per modificare il livello di autonomia dell'agente, basta cambiare quel singolo numero. Il punteggio è una stima; il ledger misura l'accuratezza delle stime.

Il ledger

Al termine di ogni subagente, l'agente principale accoda una riga a `docs/agents/ledger.txt`. Solo aggiunte; mai modificare le vecchie righe.

```
# date|agent|model|task|area|gate|necessity|outcome|tokens|tools|secs
2026-03-05|deep|opus|flaky index rebuild test|web|-|2|DONE|88000|21|420
```

Campo	Significato
date, agent, model	data, quale ruolo, quale modello
task, area	breve descrizione e scheda d'area di appartenenza
gate	1-5 = quante risposte al Readiness Gate erano SÌ; `-` se non applicabile
necessity	1-5 come dichiarato per il modello costoso; `-` per i ruoli economici
outcome	`DONE`, `BLOCKED`, `FAILED x2`, `ESCALATED`
tokens, tools, secs	numeri interi ricavati dal report di esecuzione

agent_audit.py

`python scripts/agent_audit.py` stampa, per ciascun agente, le percentuali di esito, la mediana di token e di secondi, poi la %DONE per punteggio di gate e per necessità. Segnala quando le esecuzioni con gate <= 2 falliscono più del 20% (gate troppo ottimista) e quando `deep` è stato usato con necessità <= 3. Righe non valide nel ledger escono su stderr come `line N: reason` con exit code 1.

Esempi pratici

Esempio 1: un ledger che evidenzia un uso superfluo del modello costoso

```
agent             runs   %DONE  %BLOCKED  %FAILED  %ESCALATED   tok_med  secs_med
coder                4    50.0       25.0     25.0         0.0     17050      57.5
deep                 1   100.0        0.0      0.0         0.0     88000       420
light                2   100.0        0.0      0.0         0.0     33400       145

expensive agent used 1 time(s) with necessity <= 3: the cheap path was enough on paper
```

L'esecuzione di `deep` è costata 5 volte la mediana del coder ed è stata registrata con necessità 2. Con soglia 4 non avrebbe dovuto partire senza autorizzazione; il team ora annota il punteggio prima della chiamata.

Esempio 2: calibrare la soglia con i dati

Dopo 40 righe, ogni esecuzione con necessità 3 passata prima da `planner` si è conclusa con `DONE` usando il modello economico, e le due con necessità 4 erano le uniche in cui il modello economico aveva fallito. La soglia è rimasta a 4. Se una task a necessità 3 fosse fallita due volte, la soglia sarebbe scesa a 3.

Esempio 3: intercettare una riga malformata

```
$ python scripts/agent_audit.py
line 14: gate must be 1-5 or -
$ echo $?
1
```

Un errore di battitura nella colonna gate altererebbe silenziosamente le statistiche; lo script la rifiuta e genera comunque il report sulle righe valide.

9. Consistency gate: file di specifica, deriva, divergenze

> **TL;DR** Quando un'informazione vive in più file o su più piattaforme, `consistency.py` le confronta. Un registro stabilisce cosa è promesso; un elenco di divergenze annota cosa differisce e perché; `check` fallisce in presenza di derive impreviste e `gate` elenca i blocchi prima del rilascio.

I file coinvolti

File	Contenuto
`docs/spec/consistency.toml`	quale codice confrontare: `[spec]` (costante revisione), `[config]` (chiavi lette), `[[match]]` (file di riferimento e copie)
`docs/spec/specs.toml`	revisioni e voci di specifica: id, since, stato (`implemented`, `partial`, `planned`), percorsi, piattaforme
`docs/spec/config-keys.toml`	tutte le chiavi di configurazione lette dal codice: ambito, valori, default, significato
`docs/spec/platforms.toml`	le piattaforme e le funzionalità in cui differiscono realmente
`docs/spec/divergences.toml`	ogni differenza nota: causa (`capability`, `history`, `todo`, `unknown`) e stato (`open`, `suspended`, `untested`, `resolved`)
`docs/spec/known.toml`	segnalazioni automatiche temporaneamente accettate, con tipo e motivazione

Cosa verifica `check`

- **match**: per ciascun blocco `[[match]]`, l'insieme di valori estratti tramite regex nel file di verità (es. le route del server) coincide con quello in ogni copia (client, docs).
- **spec**: la costante di revisione nel codice coincide con l'ultima revisione del registro; gli ID delle voci sono univoci; i file indicati in `where` esistono.
- **config**: ogni chiave letta dal codice è registrata con l'ambito corretto, e ogni chiave registrata viene effettivamente letta.
- **divergence**: una divergenza dovuta a una funzionalità fa riferimento a una voce esistente; cause, stati e file di prova sono validi.
- **stale**: una voce in `known.toml` non più rilevata da alcuna regola deve essere rimossa.

L'identificatore di una segnalazione accettata include i valori mancanti ed extra (`match:routes:src/mobile/routes.js:-/export`), così accettare il divario di oggi non nasconde quello nuovo di domani.

```
[[match]]
name = "routes"
truth  = { file = "src/server/app.py",    pattern = '@route\("(/[\w-]+)"' }
copies = [
  { file = "src/web/routes.js",    pattern = 'path: "(/[\w-]+)"' },
  { file = "src/mobile/routes.js", pattern = 'path: "(/[\w-]+)"' },
]
```

Il principio e il gate

Le piattaforme si comportano nello stesso modo **a meno che una capacità dichiarata differisca realmente**. Una differenza derivante da una capacità è accettata; qualsiasi altra differenza è un difetto, registrato in `divergences.toml` il giorno stesso in cui viene individuato. `consistency.py gate` esce con codice 1 finché qualsiasi divergenza `history`, `todo` o `unknown` è `open` o `suspended`, qualsiasi segnalazione accettata è di tale tipo, oppure una voce di specifica non è `implemented`. Lo stato `untested` (corretto, non ancora provato sulla piattaforma reale) non blocca ma viene stampato come promemoria "da provare". `since PATTERN FILE` individua il primo tag di release che include un pattern, per compilare `since` nel registro.

Tre numeri restano distinti: la **versione di prodotto** (ogni release), la **revisione di specifica** (ciò su cui i client possono fare affidamento; cresce sempre) e la **versione contenitore** (struttura dei file archiviati; cambia solo quando un vecchio lettore deve rifiutare il formato).

Esempi pratici

Esempio 1: una deriva individuata il giorno stesso in cui si verifica

```
DRIFT   match   routes: src/web/routes.js differs from src/server/app.py
                (missing ['/admin'], extra [])
DRIFT   match   routes: src/mobile/routes.js differs from src/server/app.py
                (missing ['/admin', '/export'], extra [])
consistency: 2 drift, 0 accepted, 1 stale
```

Uno sviluppatore ha aggiunto `/admin` solo sul server. Il controllo indica i due client che ne sono privi. Il vecchio divario accettato per `/export` risulta "stale" perché la segnalazione ha cambiato forma: deve essere riaccettata deliberatamente, oppure i client allineati.

Esempio 2: una chiave mai registrata

```
DRIFT   config      the code reads `theme` and docs/spec/config-keys.toml does not list it
```

Soluzione: aggiungere un blocco `[[key]]` con ambito, valori, default e significato. Il registro è il modo con cui documentazione e supporto scoprono l'esistenza della chiave.

Esempio 3: un difetto contro una limitazione di piattaforma

```
$ python scripts/consistency.py gate
BLOCKS RELEASE  D01 (history, priority 2): search filters: port the filter panel to the mobile client
BLOCKS RELEASE  spec item export is partial: web only: the mobile client has no export screen
TO TRY ON THE REAL PLATFORM  D03: haptics: implemented on the phone, not tried on a real device yet
gate: 2 blockers for the release
```

D02 (nessun export su smartphone) è una capacità, `no-fs-access`, quindi non blocca. D01 è solo storico: nessuno ha portato il pannello. Quello è un difetto e blocca il rilascio.

10. Audit della documentazione

TL;DR `docs_audit.py score` restituisce un unico numero per lo stato della documentazione: token sempre caricati, più la scheda d'area media, più le penalità. Minore è il valore, migliore è la qualità; zero penalità è l'obiettivo. La skill `docs-loop` lo utilizza per riordinare i documenti una modifica misurata alla volta.

Il punteggio

I token corrispondono ai byte divisi per 4. `score = token sempre caricati (CLAUDE.md + docs/progress.md) + token medi di una scheda d'area + penalità`. L'exit code è 1 in presenza di qualsiasi penalità.

Penalità	Punti	Causa
orfano	500	una scheda d'area non collegata da `docs/progress.md`
link rotto	300	un link Markdown o percorso tra backtick la cui prima cartella esiste, che punta a un file mancante
dimensione eccessiva	200	una scheda d'area con oltre circa 120 righe
duplicato	50	la medesima riga di 60 caratteri in due file tracciati
manca puntatore all'indice	2000	`CLAUDE.md` non menziona `docs/progress.md`

Comandi principali

```
python scripts/docs_audit.py score            # il punteggio e la sua scomposizione (--json per i tool)
python scripts/docs_audit.py repeats 30       # prompt/comandi ripetuti in almeno 3 sessioni
python scripts/docs_audit.py log 4480 keep "spostate regole mobile in docs/progress/web.md"
python scripts/docs_audit.py best             # miglior punteggio mantenuto finora
python scripts/docs_audit.py --root examples/demo score
```

La skill docs-loop

Un ciclo con al massimo 8 passaggi. Ogni passaggio verifica **una sola ipotesi**, in questo ordine: correggere una penalità di audit; spostare testo specifico d'area fuori da `CLAUDE.md`; spezzare o indicizzare un file riletto spesso; rimuovere una riga d'indice che non aiuta a trovare file. Copia di backup dei file toccati, modifica, rilettura del diff (le informazioni si spostano, non si perdono), misurazione, poi `log ... keep` oppure ripristino e `log ... discard`. Il ciclo si arresta dopo due scarti consecutivi. `repeats` evidenzia inoltre i workflow candidati a diventare una nuova skill.

Esempi pratici

Esempio 1: un file rinominato rompe un link

```
score 1517  (always loaded 550 + mean area 167 + penalties 800)
  ! broken link in docs/progress.md: progress/web.md
  ! orphan: docs/progress/clients.md is not linked from the index
```

`web.md` è stato rinominato in `clients.md` senza aggiornare l'indice: un link rotto e un orfano, 800 punti, rilevati prima che chiunque legga un doc non aggiornato. Corretto il link, entrambe le penalità svaniscono.

Esempio 2: un ciclo di docs-loop

Ipotesi: il paragrafo "note di design" in `docs/progress/api.md` è duplicato in `CLAUDE.md`. Backup, eliminazione della copia in `CLAUDE.md` mantenendo un puntatore, punteggio da 5.210 a 4.980. `log 4980 keep "deduplicazione note design"`. Il turno successivo non trova miglioramenti; il ciclo termina.

Esempio 3: un candidato per una skill

```
repeated commands (3 or more sessions):
    6  git fetch origin && git log
    4  python scripts/consistency.py check
```

La seconda riga è già uno script; la prima, accompagnata da una breve sintesi, diventa una skill o un prompt per `ops`. Solo le ripetizioni non coperte da skill meritano di essere formalizzate.

11. Audit della CI

TL;DR `ci_audit.py` legge il workflow di CI come testo e verifica le regole che lo mantengono economico e affidabile: un timeout su ogni job, un gruppo di concorrenza, branch di sync ignorati, attese nei test congrue con i tempi noti, comandi obbligatori presenti. Con `--gh` aggiunge durata media e tasso di successo.

I controlli eseguiti

1. Ogni job definisce `timeout-minutes`, e il workflow include un blocco `concurrency` (i job superati vengono annullati, i tag no).
2. `on.push.branches-ignore` include `sync/**`, così il push di un heartbeat non avvia mai una pipeline.
3. Nei file indicati da `test_globs`, `expect("text", timeout=T)` viene segnalato se T è inferiore all'attesa nota per quel testo (`[waits]` in `.ci-audit.toml`).
4. Ciascun comando in `required_run` compare in un passaggio `run:`.
5. Con `--gh`: `gh run list` calcola durata media e percentuale di successo delle ultime 20 esecuzioni (omesso con avviso se `gh` è assente).

```
# .ci-audit.toml
workflow     = ".github/workflows/ci.yml"
test_globs   = ["tests/**/*.py"]
required_run = ["python scripts/selftest.py"]
[waits]
"index rebuild" = 19        # secondi realmente necessari al passaggio
```

Il workflow è testo semplice analizzato per indentazione, senza librerie YAML. Conserva la struttura standard: `concurrency` a livello radice, blocco `jobs:` con figli rientrati di due spazi, e `branches-ignore: [reports, 'sync/**']` sotto `push`. Esce con codice 1 su qualsiasi anomalia; va eseguito ogni volta che si tocca `.github/workflows/` o il timeout di un test.

Esempi pratici

Esempio 1: un job privo di timeout

```
   ! job 'tests' has no timeout-minutes
ci_audit: 1 problem(s) in .github/workflows/ci.yml
```

Un test bloccato rimarrebbe in esecuzione fino al limite predefinito della piattaforma, intasando la coda. L'audit blocca il workflow finché il job non riceve un limite calibrato sulle corse reali.

Esempio 2: i push di sync avviavano la CI

Gli operatori inviano heartbeat ogni pochi minuti. Senza `sync/**` in `branches-ignore`, ciascuno scatenava un'intera pipeline CI. Il controllo ha reso visibile la svista in fase di review, e la correzione ha richiesto una sola riga.

Esempio 3: un'attesa troppo breve

```
   ! tests/e2e.py:9 waits 5 s for 'index rebuild finished', the known wait is 19 s
```

Un test attendeva 5 secondi per un'operazione che ne richiede 19, fallendo una volta su tre. La tabella in `.ci-audit.toml` codifica l'attesa misurata, così l'instabilità viene intercettata dall'audit anziché dalla terza build rossa.

12. Sincronizzazione multi-operatore

> **TL;DR** Più agenti (Claude in locale o in cloud, Antigravity e altri) collaborano in parallelo seguendo un protocollo scritto: un supervisore (SPV) pianifica e revisiona, gli operatori consegnano sui propri branch, ciascun file ha un solo scrittore, le inbox smistano le task, una riga di consegna e una di review chiudono ogni work order, e uno script segnala chi è attivo. Il protocollo vive esclusivamente sul branch `sync/<nome>`.

Ruoli, file e branch di sync

SPV scrive `SYNC.md` (contratti, work order, decisioni, Log), `sync/wo/*` e `sync/inbox/*`. Ciascun operatore scrive solo sul proprio `sync/<ID>.md`: stato, task, consegne, avanzamento, problemi, richieste. Tutti leggono tutto. Gli operatori sono elencati in `sync/operators.toml`. L'**utente** supervisiona, decide e si occupa dei merge definitivi.

Ogni progetto adotta un branch di sincronizzazione `sync/<nome>` creato dal branch principale. `SYNC.md`, `sync/*` e `tools/sync_*.py` vengono modificati e inviati solo lì (`git pull --rebase`, mai commit di merge); il branch principale mantiene i soli template, così due sessioni di sync non interferiscono mai tra loro. Il **codice** vive sui working branch (`claude/<argomento>`) e raggiunge il branch principale solo dopo `REVIEW: APPROVED` e con il consenso dell'utente. SPV opera da un worktree separato per non dover mai cambiare il proprio branch di lavoro.

Stati e blocco di stato

Stato	Significato
`WORKING {task}`	al lavoro su quella specifica task
`WAITING {chi}`	bloccato in attesa dell'utente, di un altro operatore o di un servizio
`FREE`	task concordate concluse e inviate con push; disponibile per nuovo lavoro
`DONE`	stato terminale: sessione conclusa
`IDLE`	in WORKING/WAITING da oltre 10 minuti oltre l'`eta` senza aggiornamenti (senza `eta`: 60 minuti); mai applicato a FREE o DONE

Il blocco in cima a `sync/<ID>.md` è gestito esclusivamente da `tools/sync_hdr.py`, mai a mano: `schema`, `id`, `state`, `task`, `waiting_on`, `progress`, `eta`, `wakeup`, `ack`, `updated`. Gli orari sono in formato ISO 8601 con fuso. L'`eta` è obbligatorio in WORKING e WAITING; l'`ack` è la ricevuta di lettura (l'ultima riga del Log letta).

```
python tools/sync_hdr.py ALPHA --state WORKING --task "WO-12" --eta 2026-10-01T11:30+02:00 --wakeup no
python tools/sync_hdr.py ALPHA --ack auto          # heartbeat: updated + ack, stato e task inalterati
```

Handshake, assegnazione, inbox

1. L'utente richiede "fai l'handshake `sync/<nome>`". L'operatore scarica con fetch, crea il checkout del branch in un worktree separato e si allinea con rebase.
2. Legge `SYNC.md` (ruoli, contratti, work order, regole) e `sync/operators.toml`.
3. Crea esclusivamente `sync/<ID>.md` copiando `sync/_template.md`: blocco di stato, `## Profilo` (ambiente di esecuzione e vincoli, modelli, subagenti, tool) e `## Requisiti` se la sezione 2 ne elenca.
4. Commit, `git pull --rebase`, push su `sync/<nome>`, poi attesa. **Nessun lavoro inizia prima che SPV scriva l'assegnazione.**

Ogni task e indicazione di urgenza viene recapitata in `sync/inbox/<ID>.md`: al massimo 5 righe, una sola task, con work order, branch, comando di verifica ed `eta`. L'heartbeat analizza il `git diff` di quel file, poi esegue `--ack auto`.

Consegna, revisione, sign-off

Un work order completato consiste in una riga dentro `## Consegne`, seguita da `FREE`:

```
DELIVERED WO-1b @ claude/search-filters@e4f5a6b · verify: python -m pytest tests/test_search.py -k filters
    -> 8 passed · 2026-10-01T11:10+02:00        (una sola riga nel file; qui mandata a capo)
```

`DONE` non è una consegna; un segnale non standard non attiva nulla. SPV non si fida ciecamente del report: un subagente **indipendente** esamina il commit consegnato in un worktree temporaneo: (1) perimetro rispetto ai file citati

nel work order, (2) nuova esecuzione di ogni comando di verifica, distinguendo i fallimenti preesistenti da quelli introdotti, (3) lettura del diff alla ricerca di anomalie logiche o di sicurezza, (4) verdetto per ciascun work order con citazione `file:line`. SPV scrive quindi una sola riga nel Log, `REVIEW: APPROVED` o `REVIEW: CHANGES REQUESTED`, lunga al massimo 600 caratteri. Solo dopo l'approvazione APPROVED avviene l'integrazione, sul branch di integrazione di SPV.

Sign-off: quando SPV scrive `SIGNOFF <ID>: motivo`, l'operatore non avvia nuove attività, consegna e invia con push quanto già prodotto, scrive la riga di chiusura, si imposta come `DONE` e interrompe gli heartbeat. Può rientrare solo con un nuovo handshake. Un sign-off non costituisce una colpa; il lavoro consegnato resta valido.

Heartbeat e controlli

L'heartbeat è governato dagli eventi, senza timer periodici: aggiorna il file e invia il push a ogni task completata; una task lunga viene spezzata con una riga di avanzamento; un problema o una richiesta consiste in una riga pushata (`WAITING`) più una sola frase in chat. Il costo dipende dall'esecutore:

Chi esegue l'heartbeat	Costo di un singolo heartbeat
L'agente principale (timer che ripropone il prompt)	rilegge l'intera conversazione: il più costoso
Un piccolo subagente (modello economico, contesto pulito)	circa 5-15 mila token
`tools/sync_watch.py` (senza modello AI)	0 token; risveglia l'agente solo al variare di inbox o righe indirizzate

`python tools/sync_check.py [--strict]` fornisce il quadro oggettivo da cui SPV genera i report: anzianità di ogni blocco, `eta`, `ack`, righe non lette destinate a un operatore, consegne fuori formato, inbox superiori a 5 righe, righe di Log oltre 600 caratteri e operatori IDLE. Regole fisse: un solo scrittore per file, task da 45 minuti o meno, l'agente principale resta libero, i messaggi tra operatori sono blocchi `from <ID> to <ID>: ...` in chat, le ipotesi sono etichettate `HYPOTHESIS`, e una modifica alle regole è una riga `RULE` valida per un operatore solo dopo il suo `ack`.

Esempi pratici

Esempio 1: CHANGES REQUESTED, con prove concrete

```
2026-10-01T10:40+02:00 | ALPHA | claude/search-filters@a1b2c3d | REVIEW: CHANGES REQUESTED |
WO-1: perimetro ok, 6 test passati, ma parse_filters("tag=") restituisce ogni task. Il revisore
lo ha riprodotto con un nuovo test (-k filters_empty, fallisce). Correzione in WO-1b, nella tua inbox.
Niente all'infuori di src/server/search.py e del suo test.
```

La riga indica il commit, la riproduzione e il passo successivo. ALPHA l'ha letta, ha inserito `read 10:40` nel suo avanzamento, ha consegnato WO-1b, e il nuovo test è fallito sul vecchio codice (0/1) passando sul fix, rendendo l'approvazione immediata.

Esempio 2: un blocco tra operatori evitato grazie al perimetro

ALPHA possiede `src/server/search.py`, BETA possiede `src/mobile/*`; il file condiviso delle route è nominato in un Work Order, "una riga per ciascuno". Quando entrambi hanno avuto bisogno di `src/web/routes.js` lo stesso giorno, l'ordine delle righe è stato stabilito da SPV nell'inbox anziché dall'ultimo push atterrato. Nessun conflitto, nessuna riga sovrascritta, e il diff di review ha verificato il perimetro in pochi secondi.

Esempio 3: un operatore silente individuato dal controllo

```
  ! BETA: IDLE by rule (no update for 95 min, eta 2026-10-01T11:25+02:00); tell the user
1 problem(s)
```

Il container di BETA era andato in sospensione interrompendo il job in background. `sync_check` lo ha segnalato oltre l'`eta` più 10 minuti. SPV ha avvertito l'utente in chat; un messaggio nella sessione di BETA è stato l'unico canale utile per sbloccare l'operatore, che ha ripreso nel giro di pochi minuti.

13. Scrivere regole come istruzioni positive

TL;DR Esprimi ogni regola come cosa fare, non come divieto. Dire "non fare X" mantiene l'attenzione sull'errore. Una regola ritirata si cancella, non si sostituisce con la sua negazione (che parrebbe una regola nuova). Consenti divieti espliciti solo come eccezione con danno documentato.

La regola fondamentale

`CLAUDE.md` sez. 2 riporta il principio: scrivi ogni regola come cosa fare ("fai così"); quando una regola decade, cancellala senza sostituirla con la sua negazione; aggiungi un divieto esplicito solo se l'agente ripete un danno nonostante la regola positiva, annotando il danno accanto ad esso.

Perché funziona

- Un divieto richiama l'azione da evitare: nominarla spesso la rende più probabile nell'attenzione del modello.
- Una regola positiva offre una procedura da seguire; una negativa lascia un vuoto colmato a caso.
- Cancellare una regola non lascia residui; sostituirla con un divieto trasforma la storia in vincolo.
- Un divieto con danno annotato è verificabile: sai quando rimuoverlo (quando il danno scompare).

Modelli di riscrittura

Prima	Dopo
Non modificare a mano file generati.	Modifica l'input del generatore, poi esegui `<build-cmd>`; committa entrambi.
Non leggere file di log interi.	Cerca nel log con `grep -n <pattern>`, poi leggi la porzione attorno al match.
Non tirare a indovinare se non chiaro.	Fermati subito con `BLOCKED: <la domanda esatta>`.
Non includere segreti nei commit.	Conserva le chiavi nell'ambiente o vault; il repo contiene solo i nomi.
Non usare il modello caro per compiti semplici.	Usa il modello costoso solo a necessità 4-5; sotto avvia `planner`.

L'eccezione

Se l'agente ripete un danno nonostante la regola positiva, aggiungi un divieto con il danno a fronte, per poterlo rimuovere quando cessa: "Esegui le migrazioni con `<build-cmd> migrate`. Eccezione, aggiunta 2026-05: mai eseguire SQL diretto sul database condiviso; ha azzerato i dati di staging due volte."

Esempi pratici

Esempio 1: una regola riscritta

Prima: "Non usare lo stato globale negli handler." Dopo: "Passa lo stato della richiesta tramite argomenti della funzione." L'agente ha smesso di inserire cache a livello modulo perché la regola gli indicava cosa fare.

Esempio 2: ritirare una regola in modo pulito

La regola "esegui l'importatore legacy prima dei test" è decaduta con la rimozione dell'importatore ed è stata cancellata. Un collega ha suggerito di aggiungere "non eseguire l'importatore"; questo avrebbe insegnato a ogni sessione futura che l'importatore esiste, perciò non è stata aggiunta.

Esempio 3: un'eccezione con la spiegazione del danno

Un agente faceva force push sui branch di sync per i push respinti. Oltre alla regola positiva (`git pull --rebase origin sync/<nome>`), è stata aggiunta un'eccezione: "mai forzare il push su branch di sync: ha cancellato le consegne di due operatori il 2026-10-01". Senza recidive per un mese, la riga va rimossa.

14. Bootstrap passo dopo passo

> **TL;DR** Da zero a un assetto operativo in circa un'ora: copia il kit, sostituisci i segnaposto, scrivi l'indice e le prime schede d'area, registra cosa mantenere allineato, integra il selftest nella CI e completa una prima task reale con planner e gate. Il protocollo di sync si aggiunge solo all'arrivo di un secondo operatore.

Checklist con comandi operativi

1. **Copia il kit** nel repository: `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`, `.claude/`, `docs/`, `scripts/`, `tools/`, `SYNC.md`, `sync/`, `.github/workflows/ci.yml`, `.ci-audit.toml`. Puoi omettere `examples/` se non desideri la demo, ma mantieni `scripts/selftest.py` adattandone i percorsi.
2. **Sostituisci i segnaposto**: `grep -rn '<PROJECT>\|<main-branch>\|<build-cmd>\|<test-cmd>' CLAUDE.md AGENTS.md GEMINI.md .claude` e correggi ogni occorrenza.
3. **Compila l'indice e le schede d'area**: `docs/progress.md` con una riga per area; copia `docs/progress/core.md` per ogni area; compila `How it is today`, `Open work`, `Rules`.
4. **Aggiungi la roadmap**: definisci i livelli in `docs/ROADMAP.md` con ID, Pri, Stato, Obiettivo, Requisito, Dettagli, Dopo; una sola riga è sufficiente per iniziare.
5. **Registra cosa mantenere allineato**: in `docs/spec/consistency.toml` aggiungi un blocco `[[match]]` per ciascun dato condiviso tra più file, un blocco `[config]` per le chiavi e la costante di revisione `[spec]`; compila quindi `specs.toml`, `config-keys.toml`, `platforms.toml`.
6. **Configura la CI**: assegna `timeout-minutes` a ciascun job, imposta `concurrency`, `branches-ignore: [reports, 'sync/**']`; inserisci il comando di test in `required_run` dentro `.ci-audit.toml`.
7. **Verifica il tutto**: lancia i controlli sottostanti fino a ottenere exit code 0 su tutti.
8. **Esegui una prima task reale**: descrivila a grandi linee a `planner`, ricevi i Work Order, verificali con il gate, delega a `coder` o `light`, chiudi con la checklist e registra la riga nel ledger.
9. **Aggiungi la sincronizzazione all'ingresso di un secondo operatore**: crea `sync/<nome>`, copia `SYNC.md`, `sync/`, `tools/sync_*.py` lì, compila le sezioni 2 e 3, poi invia l'istruzione "fai l'handshake `sync/<nome>`" nella sessione di ciascun operatore.

```
python scripts/selftest.py                  # il kit stesso
python scripts/docs_audit.py score          # 0 penalità
python scripts/consistency.py check         # 0 derive
python scripts/consistency.py gate          # elenco blocchi per la prima release
python scripts/ci_audit.py                  # 0 problemi
git switch -c sync/<nome>                   # solo all'arrivo di un secondo operatore
```

Cosa personalizzare per primo

- Gli invarianti nella sezione 4 di `CLAUDE.md` e le prime due regole di ogni scheda d'area: contengono ciò che un agente non può dedurre dal codice.
- I modelli indicati in `.claude/agents/*.md` qualora il proprio account adotti denominazioni differenti.
- La soglia di autonomia (sezione 3 di `CLAUDE.md`): inizia da 4, tarala in base ai dati del ledger dopo circa 30 esecuzioni.

Esempi pratici

Esempio 1: cronologia di bootstrap per Taskly

Minuti 0-10: copia del kit, sostituzione segnaposto. 10-25: `docs/progress.md`, `api.md` e `web.md` ricavati dal codice esistente. 25-40: due blocchi `[[match]]` (route, codici errore), costante di revisione, quattro chiavi di config registrate. 40-50: CI sistemata (due timeout inseriti, `sync/**` ignorato). 50-60: `selftest.py` verde, primo Work Order superato al gate. Score a 717 con zero penalità.

Esempio 2: la prima task reale

"Aggiungi la paginazione alla ricerca." `planner` ha emesso due ordini; il primo (parametro server, file e test specificati) ha superato il gate ed è andato a `coder` (`DONE`, 60 s); il secondo (UI client) ha fallito la D5 poiché la dimensione pagina non era decisa, diventando una voce in `docs/open-decisions.md`. La checklist di chiusura ha infine aggiornato `Open work` e inserito una nuova regola.

Esempio 3: aggiungere un'area in un secondo momento

```
cp docs/progress/core.md docs/progress/billing.md
# modificalo, poi aggiungi una riga alla tabella delle aree in docs/progress.md
python scripts/docs_audit.py score
```

Se ci si dimentica di aggiornare l'indice, la nuova scheda risulta orfana e lo score sale di 500 punti, rendendo immediatamente evidente l'omissione.

15. Manutenzione e metriche

TL;DR Il workflow rimane efficace solo se viene costantemente misurato e potato. Una breve routine a ogni task, milestone e mese; pochi indicatori chiave da monitorare; l'abitudine costante di eliminare regole e file che non corrispondono più al vero.

La routine operativa

Quando	Cosa fare
A ogni task	Checklist di chiusura (`CLAUDE.md` sez. 5): aggiorna `Open work` e `Rules`, esegui `docs_audit.py score`; `consistency.py check` e `ci_audit.py` quando pertinenti; registra una riga nel ledger per ciascun subagente
A ogni milestone	Cicli di `docs-loop` (al massimo 8); aggiorna la riga della ROADMAP; verifica `consistency.py gate`; risolvi o aggiorna le divergenze `untested`
A cadenza mensile	Analizza `agent_audit.py`: calibra la soglia di autonomia e il rigore del Readiness Gate; rimuovi le regole obsolete; lancia `ci_audit.py --gh`
A ogni modifica agli script	`python scripts/selftest.py`, con l'aggiunta di un controllo negativo per il nuovo comportamento
A ogni modifica al workflow	Aggiorna `docs/WORKFLOW.md` e rigenera questo PDF con `python scripts/build_workflow_pdf_it.py`

Metriche chiave da monitorare

Metrica	Origine	Valore ottimale
Score dei documenti e penalità	`docs_audit.py score`	0 penalità; token sempre caricati stabili o in calo
Derive di coerenza e blocchi al gate	`consistency.py check`, `gate`	0 derive; blocchi in diminuzione verso il rilascio
%DONE per punteggio di gate	`agent_audit.py`	gate 1 prossimo al 100%; nessun avviso di gate troppo ottimista
Uso del modello costoso a bassa necessità	`agent_audit.py`	nessun avviso; le righe di `deep` hanno necessità 4 o 5
Durata media e successo della CI	`ci_audit.py --gh`	minuti stabili; nessun job supera il proprio timeout
Operatori IDLE e righe non lette	`sync_check.py`	0 anomalie quando la sincronizzazione è attiva

Potatura

- Elimina una regola non appena cessa di essere vera; non formularne la negazione (sezione 13).
- Rimuovi le voci completate da `Open work`; rimuovi i quesiti risolti da `open-decisions.md`.
- Elimina le voci di `known.toml` che `check` segnala come obsolete; risolvi o riclassifica le vecchie divergenze `suspended`.
- Archivia un `SYNC.md` concluso in `docs/archive/` sul branch principale e chiudi il relativo branch di sincronizzazione.

Esempi pratici

Esempio 1: revisione mensile del ledger

Quaranta righe dopo, `agent_audit.py` evidenziava il 96% di `DONE` con gate 1, il 71% con gate 2, e nessuna task con necessità 3 che avesse richiesto il modello costoso. Decisione: mantenere la soglia a 4, inasprire la domanda 2 del gate per ordini che indicano "portare" senza nomi di file, e annotare entrambe le scelte in `docs/open-decisions.md`.

Esempio 2: andamento dello score

```
$ python scripts/docs_audit.py best
4480
$ python scripts/docs_audit.py score
score 4310  (always loaded 3105 + mean area 505 + penalties 700)
```

Il miglior punteggio mantenuto è 4.480 e ora compaiono 700 punti di penalità: due schede sovradimensionate. Il successivo ciclo di `docs-loop` le suddivide, riportando lo score in linea con il trend.

Esempio 3: ritirare una skill obsoleta

La skill `sd-card-flash` descriveva una procedura manuale ormai automatizzata da un job di CI. `docs_audit.py repeats` ha mostrato zero utilizzi in 60 giorni. La cartella della skill è stata cancellata, il riferimento nella scheda d'area rimosso, e `selftest.py` ha continuato a passare, provando che nulla dipendeva più da essa.